



**UNIVERSIDAD PRIVADA DOMINGO
SAVIO
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS**

Actividad 4: Second Brain Visual – Mapa Mental Interactivo con Búsqueda
Semántica

Asignatura: Programación IV

Participantes:

- Nicolas Sebastian Reguerin Meneses
- Joaquin Alessandro Felipez Rojas
- David Willy Cruz Huanca

16 de Julio de 2026

Enlace al Prototipo

<https://github.com/Joaquinf87/second-brain>

Acceso al código fuente y documentación del proyecto

1. Introducción

El presente informe describe el diseño y planificación de un proyecto denominado “**Second Brain Visual**”, una aplicación web que funciona como un cerebro digital personal. El sistema toma notas, correos electrónicos, documentos PDF y grabaciones de reuniones, y las convierte en un grafo de conocimiento interactivo en 3D, permitiendo al usuario realizar búsquedas semánticas y descubrir conexiones entre sus notas que no eran evidentes.

El proyecto se inspira en herramientas como Obsidian, pero agrega inteligencia artificial para automatizar la organización y búsqueda del conocimiento.

2. Objetivos

2.1 Objetivo General

Desarrollar un prototipo funcional (MVP) de una aplicación web que permita a los usuarios crear, organizar y explorar un grafo de conocimiento personal a partir de notas en formato Markdown, utilizando inteligencia artificial para la búsqueda semántica y la generación automática de conexiones.

2.2 Objetivos Específicos

- Implementar un editor de notas con soporte para formato Markdown y enlaces tipo Obsidian ([[wikilinks]]).
- Desarrollar una visualización en grafo 3D que represente las conexiones entre notas.
- Integrar un modelo de inteligencia artificial local para generar embeddings y realizar búsquedas semánticas.
- Implementar un sistema de ingesta que convierta PDFs, correos electrónicos y audio en notas Markdown.
- Crear un asistente de chat que permita hacer preguntas sobre el contenido del conocimiento almacenado.

3. Alcance del Proyecto

3.1 Funcionalidades del MVP

Funcionalidad	Descripción	Estado
Editor de notas	Editor Markdown con soporte wikilinks	Planificado
Grafo 3D	Visualización interactiva de conexiones	Planificado
Búsqueda semántica	Preguntas en lenguaje natural	Planificado

Funcionalidad	Descripción	Estado
Chat con IA	Asistente que responde sobre las notas	Planificado
Ingesta de PDFs	Conversión automática a Markdown	Planificado
Ingesta de correos	Extracción de emails a Markdown	Planificado
Transcripción de audio	Grabaciones a texto Markdown	Planificado

3.2 Fuera de Alcance (para este MVP)

- Autenticación de usuarios múltiples
- Sincronización en la nube
- Aplicación móvil nativa
- Edición colaborativa en tiempo real

4. Stack Tecnológico

4.1 Frontend

Tecnología	Propósito
Next.js (App Router)	Framework web fullstack
React 19	Biblioteca de componentes
TypeScript	Tipado estático
TailwindCSS	Estilos CSS utility-first
react-force-graph-3d	Visualización de grafo 3D
react-markdown	Renderizado de Markdown
gray-matter	Parsing de frontmatter YAML

4.2 Backend

Tecnología	Propósito
Next.js API Routes	Endpoints REST
ChromaDB	Base de datos vectorial local
pdf-parse	Extracción de texto de PDFs
mailparser	Extracción de correos electrónicos
mammoth	Conversión de Word a Markdown

4.3 Inteligencia Artificial

Tecnología	Propósito
Ollama	Runtime de modelos locales
nomic-embed-text	Generación de embeddings (137M params)
Phi-4 / Llama 3.3	Chat y generación de texto
Whisper (Groq API)	Transcripción de audio (gratis)

4.4 Diagrama de Arquitectura

FRONTEND (Next.js)

- Editor de notas .md
- Grafo 3D (Force Graph)
- Chat con IA / Búsqueda Semántica

↓ API

BACKEND (Next.js API)

- Vault Manager
- Embedder (Ollama)
- ChromaDB (Vector Store)

↓

OLLAMA (localhost:11434)

- nomic-embed-text (Embeddings)
- Phi-4 / Llama 3.3 (Chat)

5. Requisitos del Sistema

5.1 Hardware

Componente	Mínimo	Recomendado
RAM	8 GB	16 GB
CPU	4 cores	8 cores
GPU	No requerida	NVIDIA 6GB+ VRAM
Almacenamiento	15 GB libres	25 GB libres

5.2 Modelos de IA

Modelo	RAM Requerida	Tamaño en Disco
nomic-embed-text	2-4 GB	274 MB
Llama 3.2 3B	4-8 GB	2 GB
Phi-4 (14B)	12-16 GB	9 GB
Llama 3.3 8B	8-16 GB	5 GB

5.3 Software

- Node.js 18+
- npm o yarn
- Ollama (para modelos de IA)
- Python 3.10+ (para ChromaDB y Whisper)

6. Diseño del Sistema

6.1 Estructura del Vault

El sistema almacena las notas en un directorio local llamado “vault”, similar a la estructura de Obsidian:

```
vault/
  notas/
    reunión-marketing-q3.md
    proyectos-ia.md
    ...
  attachments/
    diagrama.png
    informe.pdf
  .obsidian/ (opcional)
```

6.2 Formato de Nota

Cada nota sigue el formato Markdown con frontmatter YAML:

```
---
title: "Reunión Q3 Marketing"
date: 2026-07-15
source: "grabacion.mp3"
tags: [reunión, marketing, q3]
type: meeting
---

# Reunión Q3 Marketing
```

Asistentes

- Juan Pérez
- María López

Puntos Clave

- Presupuesto incrementado 20%
- Nuevo foco en TikTok

Tareas

- [] Crear calendario de contenido
- [] Definir métricas KPI

6.3 Pipeline de Ingesta

PDF/Correo/Audio --> Extracción de Texto --> Limpieza --> Genera .md --> Embedding --> C

Formato	Herramienta	Salida
PDFs	pdf-parse	Texto extraído → .md
Correos (.eml)	mailparser	Asunto + cuerpo → .md
Audio	Whisper/Groq	Transcripción → .md
Word (.docx)	mammoth	Conversión → .md
URLs	readability + turndown	Scraping → .md

6.4 Flujo de Búsqueda Semántica

1. Usuario escribe una pregunta en lenguaje natural
2. El sistema genera un embedding de la pregunta usando Ollama
3. Se consulta ChromaDB con el embedding para encontrar notas similares
4. Las notas más relevantes se envían como contexto al modelo de chat
5. El modelo genera una respuesta basada en el contexto

7. Fases de Desarrollo

Fase 1: Setup Base (Día 1)

- Inicializar proyecto Next.js con TypeScript y TailwindCSS
- Instalar y configurar Ollama con modelos necesarios
- Crear estructura del vault
- Configurar ChromaDB

Fase 2: Editor de Notas (Día 2)

- Implementar editor Markdown
- Agregar soporte para wikilinks [[nota]]

- Implementar parsing de frontmatter
- CRUD de notas (crear, leer, editar, eliminar)
- Sidebar con lista de notas

Fase 3: Grafo 3D (Día 3)

- Integrar react-force-graph-3d
- Extraer nodos de las notas (wikilinks = conexiones)
- Implementar interacción (click en nodo → abrir nota)
- Colorear nodos por tags/categorías
- Controles de zoom, pan y drag

Fase 4: IA y Búsqueda Semántica (Día 4-5)

- Pipeline de embeddings: .md → Ollama → ChromaDB
- Búsqueda semántica: pregunta → embedding → query ChromaDB
- Chat con IA: contexto de notas relevantes + Ollama
- Sidebar de “notas relacionadas”

Fase 5: Ingesta de Documentos (Día 6)

- Implementar conversión de PDFs a Markdown
- Implementar extracción de correos electrónicos
- Implementar transcripción de audio
- Interfaz de subida de archivos

Fase 6: Pulido y Pruebas (Día 7)

- Pruebas de integración
- Optimización de rendimiento
- Documentación de uso
- Preparación de presentación

8. Tecnologías y Costos

Componente	Costo
Next.js, React, TailwindCSS	Gratis (MIT)
react-force-graph-3d	Gratis (MIT)
react-markdown, remark, rehype	Gratis (MIT)
gray-matter	Gratis (MIT)
Ollama	Gratis (MIT)
nomic-embed-text	Gratis (Apache 2.0)
Phi-4 / Llama 3.3	Gratis (licencia open)

Componente	Costo
ChromaDB	Gratis (Apache 2.0)
Total	\$0

9. Consideraciones Técnicas

9.1 Rendimiento

- **Embedding por nota:** ~0.2s con GPU, ~1-2s solo CPU
- **Chat (respuesta):** ~5-15 tokens/s con GPU, ~2-5 tokens/s solo CPU
- **Indexar 100 notas:** ~20s con GPU, ~3-5 min solo CPU

9.2 Seguridad

- Todos los datos se almacenan localmente
- No se envían datos a servicios externos (con modelos locales)
- Los modelos de IA corren en la máquina del usuario

9.3 Escalabilidad

- ChromaDB maneja miles de notas sin problemas
- Para decenas de miles de notas, se recomienda migrar a pgvector
- El grafo 3D maneja ~4000 nodos de forma fluida

10. Conclusiones

El proyecto Second Brain Visual representa una solución innovadora para la gestión del conocimiento personal. Al combinar un editor de notas compatible con Obsidian, una visualización en grafo 3D y inteligencia artificial para búsqueda semántica, se ofrece al usuario una herramienta poderosa para organizar y descubrir conexiones en su información.

El uso de tecnologías de código abierto y modelos de IA locales garantiza que el sistema sea completamente gratuito y respete la privacidad del usuario, ya que todos los datos permanecen en su máquina.

El enfoque de desarrollo iterativo en fases permite entregar un MVP funcional en tiempo razonable, con posibilidad de expandir funcionalidades en el futuro.

11. Referencias

- Ollama. (2026). Ejecutar modelos de IA localmente. <https://ollama.com>
- React Force Graph 3D. (2026). Biblioteca de visualización de grafos. <https://github.com/vasturiano/react-force-graph-3d>

- ChromaDB. (2026). Base de datos vectorial. <https://www.trychroma.com>
- Obsidian. (2026). Aplicación de notas con grafo de conocimiento. <https://obsidian.md>
- nomic-embed-text. (2026). Modelo de embeddings. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>